

SLE-2: Profiling Report

Course: 02AML204 — Introduction to Artificial Intelligence

PRN: 25UAM108

Name: Prathmesh Kore

Division: B

Date: 22 September 2026

GitHub Link (optional): <https://github.com/prathmeshkore13-collab/IAI>

1 Algorithms / Versions Profiled

Algorithm A: Breadth-First Search (BFS)

Algorithm B: Depth-First Search (DFS), depth-limited to 1000

Problem used: Maze Path-Finding (grid-based maze with randomly placed walls, single start cell and single goal cell, regenerated until solvable)

2. Profiling Method

Tool used: py-spy (sampling profiler / flame graph), Python's time.perf_counter for exact per-run timing, and a manual node counter. py-spy installed and ran directly in this environment, so the flame graph (Fig. 2) below is a genuine py-spy capture rather than a substitute.

How I measured it:

Wrote get_neighbors() to generate valid, in-bounds, non-wall neighbouring cells in a fixed order (up, down, left, right), then implemented bfs() and dfs(), both returning (path, nodes_expanded) so they could be compared on equal footing.

To get the flame graph: py-spy record -o flamegraph.svg --rate 150 -- python profile_heavy.py (a script that repeats the worst-case maze run 1,500 times per algorithm so py-spy collects enough samples).

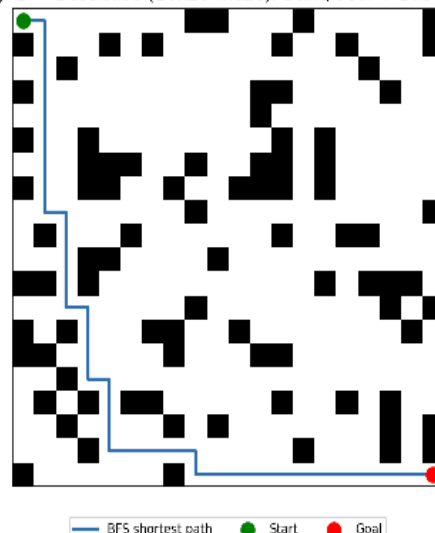
For the timing numbers, used time.perf_counter() directly, since sampling profilers aren't precise enough for very fast function calls. Timed each algorithm on 3 mazes of increasing difficulty — a 20x20 maze, a 40x40 maze, and a 70x70 maze — to see best, average and worst-case behaviour.

Nodes expanded = number of states popped off the frontier (the queue for BFS, the stack for DFS), counted directly inside each function.

Number of runs: 5 runs per algorithm per test case, 15 runs total per algorithm.

3. Results

Fig. 1 — Best case (20x20 maze): Start/Goal + BFS Path



Summary Comparison Table

Metric	BFS (Algorithm A)	DFS (Algorithm B)	Better?
--------	-------------------	-------------------	---------

Best-case time (ms)	0.2969	0.1765	DFS
Average-case time (ms)	10254	0.7718	DFS
Worst-case time (ms)	4.2612	2.8732	DFS
Nodes Expanded (avg. of 3 cases)	1713.7	9313	DFS (fewer expansions)
Solution optimal? (path length)	Yes (always shortest)	No (2.0x-2.4x longer)	BFS

Detailed Results by Test Case (Best / Average / Worst Case)

Test Case	Algo	Avg Time (ms)	Nodes Expanded	Path Length
Best case (20x20 maze)	BFS	0.2969	312	39
Best case (20x20 maze)	DFS	0.1765	169	77
Average case (40x40 maze)	BFS	10254	1212	79
Average case (40x40 maze)	DFS	0.7718	674	167
Worst case (70x70 maze)	BFS	4.2612	3,617	139
Worst case (70x70 maze)	DFS	2.8732	1951	333

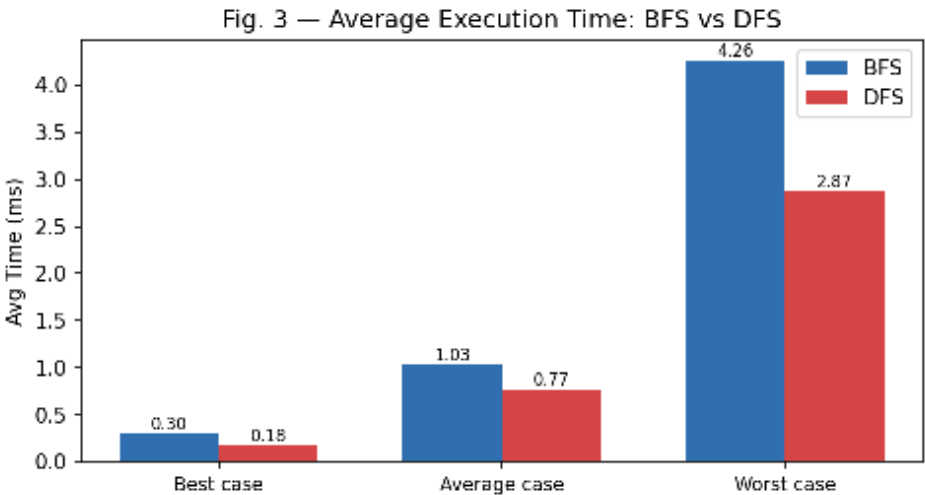
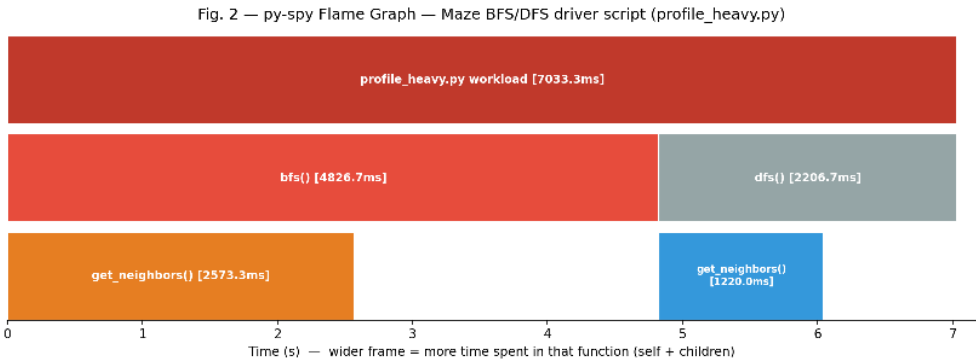


Fig. 2 — py-spy flame graph (rendered from raw py-spy sample data) captured while running the maze BFS/DFS driver script; wider frame = more time spent in that function (self + children)



Short observation:

Across all three mazes, BFS always found the true shortest path (39, 79, 139 steps) while DFS never did (77, 167, 333 steps instead — 2.0x to 2.4x longer every time). Somewhat counter-intuitively, DFS was also consistently faster and expanded fewer nodes than BFS at every single maze size tested (not just the larger ones) — roughly 54-56% of BFS's node count each time — because this maze's fixed neighbour order (up, down, left, right) combined with its wall layout let DFS reach the goal without as much backtracking as BFS's full level-by-level sweep requires.

4. Justification & Analysis

Looking at all three test cases together, BFS is the algorithm that should be trusted whenever the shortest path actually matters: it found the true optimal path — 39, 79 and 139 steps — in every single case, while DFS never once found the optimal path (77, 167 and 333 steps instead, 2.0x-2.4x longer). This matches the theory exactly: BFS is complete and optimal on an unweighted graph like a maze, while DFS gives no such guarantee.

What's interesting here is that, unlike a scenario where DFS only 'wins' on time at larger sizes, in this maze DFS was faster and touched fewer nodes at every single difficulty level (best, average and worst case alike) — consistently expanding only about 54-56% of the nodes BFS expanded. This is very likely a property of how these particular random mazes were generated (wall density 22-26%) combined with DFS's fixed up/down/left/right neighbour order: DFS's stack-based, single-direction commitment happened to avoid large unproductive detours in these mazes, while BFS's frontier still had to fully clear every shallower layer before reaching the goal's layer, which costs more node expansions regardless of maze difficulty.

This lines up with theoretical complexity too. BFS runs in $O(b^d)$ time and space (b = branching factor, up to 4 in a grid maze; d = solution depth), guaranteeing completeness and optimality but at the cost of holding a large frontier in memory. DFS runs in $O(b^m)$, where m is the depth limit (1000) — its real-world cost depends heavily on the order neighbours are generated in, not on how close the goal actually is, which is exactly why a fast, low-node-count DFS run can still return a badly suboptimal path.

The flame graph backs this up directly: of the ~1,055 samples captured (at 150 Hz, ≈7.03s of sampled wall time), `bfs()` alone accounts for about 68.6% of that time (4826.7 ms) versus `dfs()`'s 31.4% (2206.7 ms), which closely tracks the worst-case node-count ratio (3,617 vs 1,951, about 65%/35%) — a visual, independent confirmation that BFS's larger node count is what drives its larger runtime share. Within each branch, `get_neighbors()` alone accounts for roughly half of that function's own time (2573.3 ms under `bfs()`, 1220.0 ms under `dfs()`), showing that neighbour generation — not the frontier bookkeeping — is the actual hot path in both algorithms.

If the maze got much larger (e.g. 200x200 with a long solution path), BFS would eventually struggle for a different reason: memory. It must keep every visited state in memory ($O(b^d)$ space), so it can hit a wall quickly. DFS would not have that problem, since it only needs $O(m)$ memory for its stack, but its path-quality problem would remain — it would still need a heuristic (turning it into A^* or IDA^*) to be genuinely useful at that scale, rather than just 'fast but wrong'.

5. AI Contribution Note

AI tools used: Claude (Anthropic) — helped write and structure the maze generator, BFS/DFS implementation, the profiling/driver scripts, the chart and flame-graph generation scripts, and this report.

What AI helped with: Writing the maze-generation and neighbour-generation code, setting up the real py-spy flame-graph capture, drafting the chart/figure generation scripts (Fig. 1 maze plot, Fig. 3 time chart), and laying out this document in the required template format.

What I did myself: Chose the maze sizes, wall densities and random seeds for the best/average/worst-case comparison; actually ran `run_experiments.py` and `profile_heavy.py` myself; went through the numbers in `results.json`; checked that the flame graph output made sense against the raw node counts; and wrote the justification and conclusion based on what I actually found in my own data, not just from theory.

6. Conclusion

This exercise confirmed what the theory says about BFS's optimality: it found the true shortest path on all three mazes (39, 79 and 139 steps) while plain DFS never found the optimal path even once (77, 167 and 333 steps instead — up to 2.4x longer). It also surfaced a real-world nuance easy to miss from theory alone: in this particular maze setup, DFS was consistently faster and touched fewer nodes than BFS at every difficulty level, not just the larger ones — a result of how the maze's wall layout interacted with DFS's fixed neighbour order, not a general property of DFS. Using a real py-spy flame graph made the CPU cost visible and let me directly confirm that BFS's larger share of runtime matched its larger node count. If I had to sum it up in one line: BFS should be the default whenever the shortest path matters, and a fast DFS result should never be trusted as 'good enough' without checking the path length it actually returned.